

Лабораторная работа

Командные сценарии

Методические указания

1. Цель работы

Получение навыков практической работы в ОС QNX Neutrino. Изучение командных сценариев в ОС QNX.

2 Командные сценарии

Любые программы, стандартные и собственные, входящие в состав операционной системы и доступные через вычислительную сеть, можно комбинировать произвольным образом с помощью перенаправления ввода/вывода, конвейеров и других возможностей командных интерпретаторов. Если для выполнения задачи нужно выполнить несколько команд, то удобно записать эти команды в текстовый файл, а затем указать интерпретатору *shell* "выполнить" этот файл. Такой файл называют командным сценарием (*shell script*). По сути, интерпретатор реализует язык программирования высокого уровня. С помощью этого языка администраторы могут осуществлять управление вычислительной системой (что, в общем-то, они и делают).

2.1 Запуск сценариев на исполнение

С помощью любого текстового редактора создадим файл, скажем, *script0.sh*. В него поместим одну строку:

```
echo Hello world!
```

Примечание. Последним символом в командном сценарии должен быть символ перевода строки. Чтобы интерпретатор *shell* выполнил команду, введенную в командной строке, нужно нажать клавишу **<Enter>**. Так что когда у вас не будет работать какой-нибудь сценарий, не поленитесь проверить его на наличие символа перевода строки в конце файла - помогает в 50% случаев.

Четыре способа запуска сценария:

- запустить интерпретатор, указав ему имя сценария в качестве аргумента или связав стандартный поток ввода интерпретатора с файлом сценария. Например:

```
sh script0.sh
```

- запустить интерпретатор, связав стандартный поток ввода интерпретатора с файлом сценария. Например:

```
sh < script0.sh
```

- выполнить сценарий в текущем экземпляре *shell* командой "точка" (*.*). Например:

```
.script0.sh
```

- установить для сценария файловый атрибут "исполняемый" и запустить как обычную утилиту. Например:

```
chmod 755 script0.sh
```

```
script0.sh
```

Самым распространенным является последний способ. Однако, в этом случае файл сценария должен находиться в одном из каталогов, перечисленных в переменной окружения *PATH*, либо для его запуска надо указывать полное путьевое имя сценария.

Команда "точка" тоже достаточно важна. Например, встраиваемая система управления базами данных реального времени *Empress* может использовать

большое количество переменных окружения, особенно нужных при разработке приложений. Производитель поставляет командные сценарии, задающие значения этих переменных. Если запустить такой сценарий первым или третьим способом, то произойдет вот что: запустится новый экземпляр *shell*, выполнит команды из сценария и благополучно завершится, не изменив ничего в системе. А если применить команду "точка", то в нашем рабочем экземпляре *shell* будут установлены нужные переменные, позволяя нам наслаждаться инструментами *Empress*.

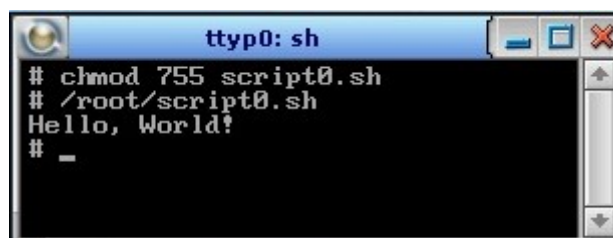
Существует много командных интерпретаторов. Язык программирования каждого из них имеет свой синтаксис и свои особенности реализации. Как, например, при третьем способе запуска будет определяться интерпретатор, необходимый для выполнения сценария? Для этой цели в самой первой строке сценария помещают символы `#!` и, без пробела, имя интерпретатора, для которого написан сценарий, например:

```
#!/bin/bash
```

Примечание. Такую конструкцию можно поместить только в первую строку. Во всех остальных случаях первый же символ `#` считается началом комментария, заканчивающегося символом конца строки.

Модифицируем наш файл *script0.sh*:

```
#!/bin/ksh
echo Hello, World! # This is comment
# This is comment too
```



2.2 Переменные и параметры сценариев

Ярким и типичным примером переменных, используемых интерпретатором *shell*, являются переменные системного окружения. Аналогичным способом можно задавать собственные переменные для сценариев или модернизировать существующие. Имя переменной *shell* выбирается так же, как это принято в большинстве языков программирования, т. е. это может быть последовательность букв, цифр и символов подчеркивания, начинающаяся с буквы или символа подчеркивания.

Примечание. Обычно имена переменных для *shell* формируют из последовательности заглавных букв в сочетании с символами подчеркивания, но это просто необязательная традиция.

Переменные в *shell* задаются сразу со значением (можно с пустым). При объявлении переменной и она, и ее значение должны быть записаны без пробелов относительно символа равенства (=).

Примечание. Пробелы возле символа `=` - типичная ошибка начинающих *shell*-программистов.

Например, создадим переменную *MY_VAR* со значением *aaaa*:

```
MY_VAR=aaaa
```

или переменную *MY_VAR1* без значения:

```
MY_VAR1=
```

В отличие от обычных языков программирования, значение *shell*-переменной - всегда строка символов. То есть команда:

```
MY_VAR2=123
```

создает переменную *MY_VAR2*, значением которой является строка *ASCII*-символов "123", но не число 123!

Если для переменной задается значение, содержащее пробелы, то нужно заключить его в кавычки:

```
MY_VAR3="QNX Neutrino RTOS v.6.3.0"
```

Чтобы получить доступ к значению переменной, надо перед ее именем поставить символ *\$*. Например, чтобы с помощью команды *echo* вывести на экран содержимое переменной *MY_VAR3*, надо выполнить такую команду:

```
echo $MY_VAR3
```

Результат выполнения команды можно присвоить переменной в качестве значения, например:

```
DATE='date'
```

При этом сначала выполнится команда *date*, а результат ее выполнения вместо стандартного вывода приписывается в качестве значения переменной *DATE*.

Если нужно, чтобы имя переменной не сливалось с другими символами, то имя переменной заключают в фигурные скобки. Так команда:

```
echo ${MY_VAR3}beta1
```

выведет сообщение:

```
QNX Neutrino RTOS v.6.3.0beta1
```

Если требуется, чтобы сценарий запрашивал ввод значений переменных, используется команда *read*. Она обеспечивает работу со сценарием в диалоговом режиме, например:

```
read A
```

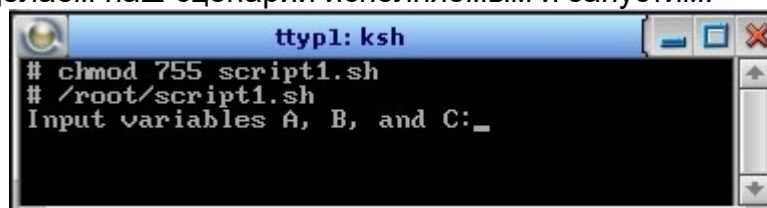
По этой команде сценарий остановится, ожидая ввода. Пользователь должен ввести какое-либо значение и нажать клавишу *<Enter>*. Введенное значение будет присвоено переменной *A* и сценарий продолжит выполнение. Таким способом можно присвоить значения сразу нескольким переменным, например:

```
read A B C
```

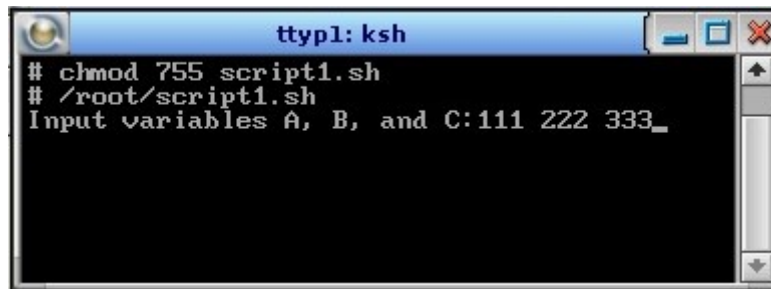
Для того чтобы попрактиковаться, напомним сценарий *script1.sh*:

```
#!/bin/ksh
echo -n Input variables A, B, and C:
read A B C
echo A=$A
echo B=$B
echo C=$C
```

Затем сделаем наш сценарий исполняемым и запустим:

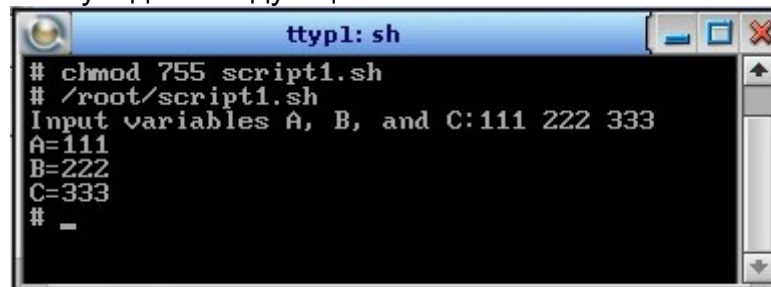


На запрос сценария введем строку *111 222 333*.



```
# chmod 755 script1.sh
# /root/script1.sh
Input variables A, B, and C:111 222 333_
```

На экране мы увидим следующее:



```
# chmod 755 script1.sh
# /root/script1.sh
Input variables A, B, and C:111 222 333
A=111
B=222
C=333
# _
```

Снова запустите сценарий, но введите строку `111 222 333 444 555` и посмотрите, как изменится результат.

Примечание. Для того чтобы можно было использовать служебные символы в составе строковых переменных, применяется экранирование - обратный слеш (`\`).

Если, скажем, переменной `A` надо присвоить в качестве значения строку `$B`, то это делается так:

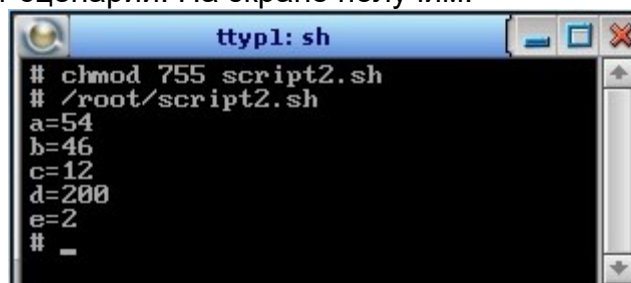
```
A=' $B '
```

Тогда команда `echo A=$A` выведет на экран: `A=$B`.

Переменные в интерпретаторе `shell` имеют строковый тип, однако `shell` предоставляет некоторые средства, позволяющие обрабатывать переменные как целые числа. Для этого предназначена команда `expr`. Напишем сценарий `script2.sh`:

```
#!/bin/ksh
x=50 y=4
a=`expr $x + $y`; echo a=$a
b=`expr $x - $y`; echo b=$b
c=`expr $x / $y`; echo c=$c
d=`expr $x "*" $y`; echo d=$d
e=`expr $x % $y`; echo e=$e
```

Запустим этот сценарий. На экране получим:



```
# chmod 755 script2.sh
# /root/script2.sh
a=54
b=46
c=12
d=200
e=2
# _
```

В результате деления получаем только целую часть результата. Остаток от деления получается с помощью операции `%`. Символ умножения (звездочка - `*`) взят в кавычки, т. к. `*` - служебный символ `shell`, обозначающий произвольную строку.

Примечание. Переменные и знаки операций **обязательно разделяются пробелами**. Отсутствие таких пробелов - очень распространенная ошибка в

shell-программировании.

Оператор выполняющий арифметические операции заключен не в обычные кавычки типа `'`, а в ``` (на русской раскладке на этой клавише значится «ё», на англ «~» и «`»).

Все переменные доступны только в том процессе *ksh*, в котором они были объявлены. Для того чтобы к переменным можно было обратиться из процессов, дочерних по отношению к данному процессу *ksh*, переменные следует "экспортировать". Это можно сделать двумя способами. Первый - создать переменную, а затем в любое время экспортировать ее:

```
VAR1=myvar
```

```
...
```

```
export VAR1
```

Второй способ - экспортировать переменную сразу при создании:

```
export VAR1=myvar
```

Посмотреть, какие из переменных данного интерпретатора экспортированы, можно, выполнив команду *export* без аргументов.

Примечание. Все переменные интерпретатора *shell* (экспортированные и не экспортированные) выводятся на экран командой *set* без аргументов.

2.3 Внутренние переменные shell

Интерпретатор *shell* поддерживает некоторые переменные, которым он присваивает значения самостоятельно (табл. 1).

Таблица 1. Внутренние переменные интерпретатора

Переменная	Значение
?	Код завершения последней команды (0 - нормальное завершение)
\$	<i>PID</i> текущего процесса <i>ksh</i>
!	<i>PID</i> последнего процесса, запущенного в фоновом режиме
#	Число параметров, переданных в <i>shell</i>
*	Перечень параметров <i>shell</i> как одна строка
@	Перечень параметров <i>shell</i> как совокупность слов
-	Флаги, передаваемые в <i>shell</i>

Примечание. Различие между *\$** и *\$@* заключается в том, что если *\$** имеет значение, например *"111 222 333"*, то *\$@* будет иметь значение *"111" "222" "333"*.

2.4 Параметры сценариев

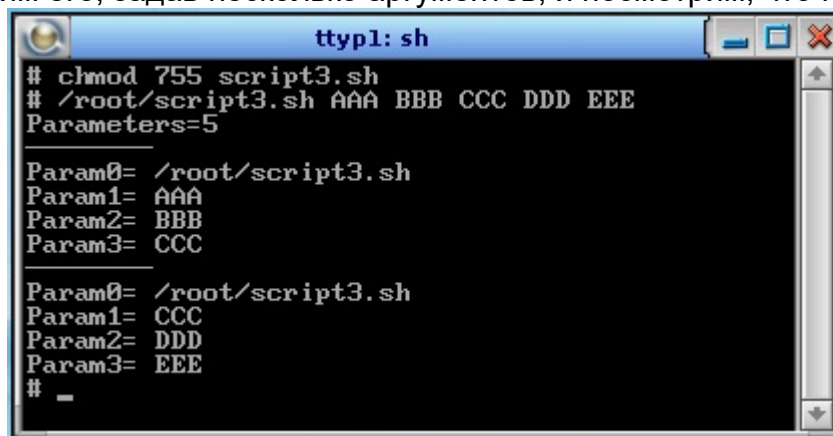
Как и любая программа, сценарий может принимать аргументы. Для этой цели *shell* поддерживает специальные переменные, имена которых состоят из одной цифры в диапазоне от 0 до 9. При этом переменная 0 содержит имя самого сценария, переменная 1 - первый параметр, переменная 2 - второй параметр и т. д. Общее число параметров содержится в переменной *#*.

А если вдруг нужно передать сценарию, скажем, 30 аргументов? Передать-то, понятно, передадим - дело немудреное, но как получить к ним доступ? А для этого есть встроенная команда *shift*. Чтобы понять принцип ее использования, вспомните, как вы просматриваете длинный документ *Word*: в один момент времени можно видеть только фрагмент текста, соответствующий рабочему

пространству окна *Word*; чтобы посмотреть другие части документа, нужно перемещать рабочее пространство окна вверх или вниз по документу ("прокручивать" документ). Аналогично, переменные 1-9 являются этим самым "окном" для просмотра аргументов. Перемещается это окно с помощью команды *shift* на нужное число позиций. Переменная 0 всегда содержит имя сценария. Для наглядности рассмотрим пример *script3.sh*:

```
#!/bin/ksh
echo "Parameters=${#*}"
echo -----
echo "Param0=" $0
echo "Param1=" $1
echo "Param2=" $2
echo "Param3=" $3
echo -----
shift 2
echo "Param0=" $0
echo "Param1=" $1
echo "Param2=" $2
echo "Param3=" $3
```

Запустим его, задав несколько аргументов, и посмотрим, что получилось:



```
tty1: sh
# chmod 755 script3.sh
# /root/script3.sh AAA BBB CCC DDD EEE
Parameters=5

Param0= /root/script3.sh
Param1= AAA
Param2= BBB
Param3= CCC

Param0= /root/script3.sh
Param1= CCC
Param2= DDD
Param3= EEE
# _
```

2.5 Отладка сценариев

Для запуска сценария в подобии отладочного режима предназначена команда *set*. Есть два режима, представляющих интерес с точки зрения отладки.

- Первый режим - устанавливается командой *set -v*, указывающей интерпретатору выводить все строки, которые он считывает; отменяется командой *set +v*.

- Второй режим (при отладке используется чаще) - устанавливается командой *set -x*, указывающей интерпретатору выводить на экран команды перед их выполнением; отменяется командой *set +x*.

2.6 Операторы языка программирования Korn Shell

Как же можно написать полноценную программу, не используя циклы и проверку условий? Язык программирования *Korn Shell* поддерживает ряд специальных операторов (команд), основные из которых:

```
test
if
```

case
for
while
until

Оператор *test*, или *[]*

Оператор *test* проверяет выполнение условия. Этот оператор в сценариях сам по себе не встречается, зато в сочетании с оператором *if* он очень распространен. Результат выполнения оператора имеет значение "истина" или "ложь". Это значит, что если требуется использовать оператор *test* в интерактивном режиме, то его результат будет содержаться в переменной ? (0 - "истина", 1 - "ложь"). У оператора два равнозначных формата, причем в равной степени в сценариях используются оба:

test условие

или

[условие]

Примечание. Между квадратными скобками и условием обязательно должны быть пробелы. Отсутствие этих пробелов - очень распространенная ошибка.

Условия могут быть:

проверка файлов;

сравнение строк;

сравнение целых чисел.

Некоторые условия проверки файлов:

test -f myfile.txt - "истина", если *myfile.txt* - обычный файл;

test -d mydir - "истина", если *mydir* - каталог;

test -c myfile - "истина", если *myfile* - символьное устройство;

test -r myfile.txt - "истина", если *myfile.txt* доступен для чтения;

test -w myfile.txt - "истина", если *myfile.txt* доступен для записи;

test -s myfile.txt - "истина", если *myfile.txt* не пустой.

Некоторые условия сравнения строк:

test aaa=bbb - "истина", если строка *aaa* совпадает со строкой *bbb*;

test aaa!=bbb - "истина", если строка *aaa* не совпадает со строкой *bbb*;

test -n \$A - "истина", если переменная *A* задана (не пустая);

test -z \$A - "истина", если *A* - пустая (т. е. не имеет значения).

Оператор *test* возвращает значение "истина", если условие - непустая строка.

POSIX определяет два стандартных "условия", которые можно использовать вместо оператора *test*, - *true* и *false*.

Некоторые условия сравнения целых чисел:

test x -eq y - "истина", если *x=y*;

test x -ne y - "истина", если *x!=y*;

test x -gt y - "истина", если *x>y*;

test x -ge y - "истина", если *x>=y*;

test x -lt y - "истина", если *x<y*;

test x -le y - "истина", если *x<=y*.

В одном операторе *test* можно задать несколько условий, комбинируя их с помощью логических операций:

! - НЕ

-o - ИЛИ

-a - И

Оператор *if*

В общем случае этот оператор условия имеет такой синтаксис:

```
if условие  
    then список_операторов [elif условие  
        then список_операторов]  
        [else список_операторов]  
fi
```

где *условие* - любой оператор, возвращающий значение *true* или *false* (обычно используется оператор *test*); *список_операторов* - операторы, разделенные точкой с запятой (;). Оператор *elif* - это сокращение от "*else if*". В квадратные скобки взяты необязательные элементы конструкции. Конструкция оператора *if* заканчивается служебным словом *fi* ("*if*" наоборот).

Проиллюстрируем сказанное простейшим примером - сценарием *script4.sh*:

```
#!/bin/ksh  
if [ -f /tmp/myfile ]  
    then cksum /tmp/myfile  
    else ls /etc > /tmp/myfile; cksum /tmp/myfile  
fi
```



Оператор *case*

Оператор выбора *case* имеет такой синтаксис:

```
case строка in  
    шаблон) список_операторов;  
    шаблон) список_операторов;;  
    ...  
esac
```

где *строка* - переменная или последовательность символов, которая последовательно сравнивается с шаблоном *шаблон*. Как только *строка* и *шаблон* совпадут, выполняется соответствующий *список_операторов*. Как, вероятно, вы уже поняли, *in* и *esac* - это служебные слова, причем завершает всю конструкцию *esac* ("*case*" наоборот).

Примечание. Список операторов для каждого шаблона заканчивается двойной точкой с запятой ;; (в аналогичной конструкции языка Си второй точке с запятой соответствует служебное слово *break*).

Пробелы в скобках в операторе *if* необходимы для корректной работы скрипта:

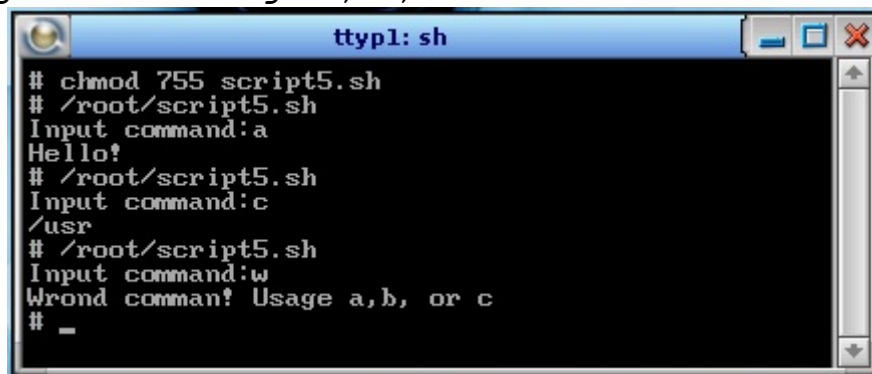
```
if [ -f /tmp/myfile ]
```

Проиллюстрируем сказанное простейшим примером - сценарием *script5.sh*:

```
#!/bin/ksh
echo -n "Input command:"
read Command
case $Command in
    a) echo Hello!;;
    b) ls -l /usr/bin;;
    c) cd /usr; pwd;;
    *) echo "Wrond comman! Usage a,b, or c";;
esac
```

Шаблону * строка будет соответствовать всегда (в языке Си для этой цели используется служебное слово *default*), то есть если будет введен символ не предусмотренный нами в case тогда мы выводим следующее сообщение:

Wrong command! Usage a, b, or c



```
# chmod 755 script5.sh
# /root/script5.sh
Input command:a
Hello!
# /root/script5.sh
Input command:c
/usr
# /root/script5.sh
Input command:w
Wrond comman! Usage a,b, or c
# _
```

Оператор *for*

Оператор цикла *for* имеет такой формат:

```
for имя [in список_значений]
do
    список_операторов
done
```

Оператор работает так. На каждой итерации цикла переменной *имя* последовательно присваиваются значения из списка *список_значений* и выполняются команды из списка *список_операторов*; *список_операторов* заключен между служебными словами *do* и *done*.

Хрестоматийный пример использования оператора *for* - сценарий *script6.sh*:

```
#!/bin/ksh
LIST=`ls /etc`
for i in $LIST
do
    if [ ! -d $i ]
    then cksum /etc/$i
    fi
done
```

В результате выполнения этого сценария на экран будет выведена информация с размером и контрольными суммами содержимого каталога */etc* (без учета содержащихся в нем подкаталогов).

```
ttty1: sh
# chmod 755 script6.sh
# /root/script6.sh
529859647      31 /etc/TIMEZONE
1631902077     677 /etc/acl.conf
2958805751     120 /etc/autocconnect
1707029532     2304 /etc/bootptab
3925038129     352 /etc/config
4205377473     970 /etc/context.conf
1521008708      4 /etc/country
1635251318     272 /etc/default
606393517      3247 /etc/dhcpd.conf
2356098930     352 /etc/enum-usb.conf
3081538614     140 /etc/ftpchroot
3884450240     401 /etc/ftpusers
1632244136     214 /etc/group
2208048213      98 /etc/hosts
91697164       2677 /etc/inetd.conf
2002233147     1300 /etc/mcd.conf
4078633282     716 /etc/mcd.mnt
643648713     477697 /etc/mib.txt
1414228391    111892 /etc/moduli
3287738687      25 /etc/motd
3390628586     552 /etc/netconfig
2431180465      37 /etc/networks
1409477367     170 /etc/nsswitch.conf
3632589844     240 /etc/ntp
3488002585     304 /etc/openssl
2951011614     2813 /etc/party.conf
2330437313     343 /etc/passwd
1258587079     1109 /etc/pf.conf
2862426396    26580 /etc/pf.os
833402465      624 /etc/photon
2460156913     400 /etc/printers
3151314964     1517 /etc/profile
1554939322     352 /etc/profile.d
2881330069     1635 /etc/protocols
1564026485     424 /etc/qnx
624557423      15 /etc/qversion
3151725953     656 /etc/rc.d
4288190226     68 /etc/resolv.conf
1610527063     743 /etc/rpc
2088548851     4125 /etc/services
475082486      240 /etc/skel
276670272      761 /etc/smpd.conf
1805323366     312 /etc/ssh
4107092225     94 /etc/syslog.conf
380162946      920 /etc/system
569460352     6749 /etc/termcap
2996814486     240 /etc/timezone
4174073322     4113 /etc/umass-enum.cfg
3982297688     642 /etc/view.conf
# _
```

Если в операторе *for* нет конструкции *in список_значений*, то *список_значений* будет равен внутренней переменной *@*.

Оператор *for* не подходит для организации бесконечных циклов, т. к. он работает, перебирая элементы конечного списка.

Операторы *while* и *until*

В отличие от оператора *for*, оператор *while* позволяет организовывать бесконечные циклы, т. к. следующая итерация инициируется всегда, если только условие истинно:

```
while условие
do
    список_операторов
done
```

В этом операторе можно задавать счетчики циклов, параметры выхода из цикла (служебное слово *break*) и т. д. Говоря кратко, оператор *while* соответствует традиционным представлениям о нем. Если *список_операторов*

оператора *while* выполняется тогда, когда *условие* истинно, то в операторе *until* напротив - когда *условие* станет истинным, цикл завершится:

```
until условие
do
    список_операторов
done
```

Разумеется, цикл можно принудительно прервать командой *break*.

Пустой оператор

Иногда по правилам синтаксиса в сценарии на каком-то месте должен обязательно стоять оператор, а никаких действий предпринимать не хочется. В таком случае используется пустой оператор "двоеточие" (:).

2.7 Функции в сценариях

Как и в любом языке программирования, в интерпретаторе *shell* можно описывать функции и затем вызывать их из любого места сценария. Определить функцию очень просто:

```
имя ()
{
    список_операторов
}
```

В любом месте программы, где нужно вызвать функцию, просто указывается ее имя как команда. Функции можно передавать аргументы так же, как аргументы передаются сценарию. Мало того, доступ к аргументам выполняется в функции так же, как в сценарии, т. е. через переменные 0—9. Другими словами, внутри функции переменные 0—9 рассматриваются как аргументы функции, а в остальных местах сценария - как аргументы сценария. Разумеется, при вызове функции новый процесс не создается. Если функция должна вернуть код завершения, то используется оператор *return*.

2.8 Обработка сигналов

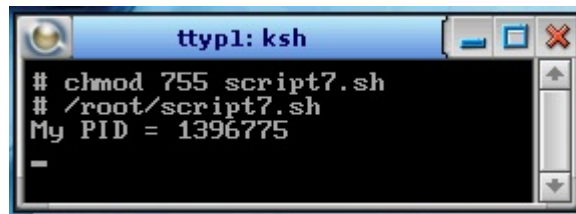
Для обработки сигналов в *shell* предусмотрен оператор *trap*. Его синтаксис:

```
trap "список_операторов" список_сигналов
```

По сути дела, *список_операторов* - это своего рода обработчик сигналов *список_сигналов*.

Например, создадим обработчик для сигнала *SIGUSR1* (с номером 16). Пусть при получении этого сигнала сценарий *script7.sh* уничтожит содержимое каталога */tmp*, выведет некоторое сообщение на экран и завершится с кодом 100:

```
#!/bin/ksh
echo My PID = $$
trap "rm /tmp/* -R; echo Good Bye!; exit 100" 16
while true
do
done
```

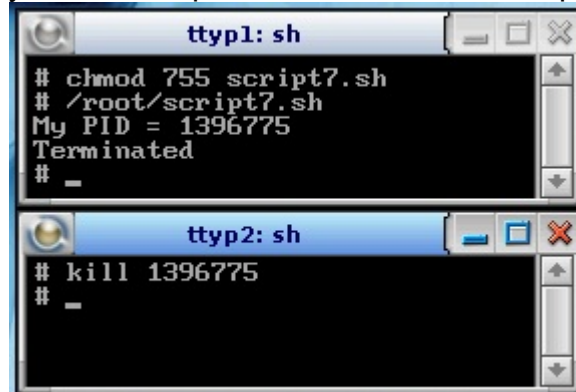


```
# chmod 755 script7.sh
# /root/script7.sh
My PID = 1396775
_
```

Для проверки запустите этот сценарий. Он выведет на экран *PID* интерпретатора, в котором он выполняется (в примере *1396775*). Затем из другого интерпретатора (с другой виртуальной консоли или из другого псевдотерминала) выполните команду:

kill -s SIGUSR1 1396775

Посмотрите результат на терминале исходного интерпретатора.



```
# chmod 755 script7.sh
# /root/script7.sh
My PID = 1396775
Terminated
# _

tty2: sh
# kill 1396775
# _
```